

deer manual

**An Independent Project on the emulation of the
Dartmouth Time-Sharing System - the DTSS emulator**

Antonio Maschio

Independent Project

31 July 2026

ABSTRACT

Introducing **deer**, the emulator of the Dartmouth Time-Sharing System (DTSS).

This text is not a primer about Operating systems; I assume you have the most common knowledge about console typing, Operating Systems and the DTSS; this text is more like a reference document that helps in using correctly the various features, their limits and enhancements.

Even though the bulk of the work is mine, it's undeniable that the Dartmouth documents available at bitsavers.org were of immense utility, and constitute the guide of this work.

Nonetheless, coding is my sole responsibility, so any error in the code is due to me and me only. Readers are encouraged to report all errors and inconsistencies (both in the program and in this text) to:

`ing dot antonio dot maschio at gmail dot com`

Support for my efforts is appreciated. Donations can be made via PayPal to

`tbin at libero dot it`

Thanks.

*Tali eravamo tutti e tre allotta,
io come capra, ed ei come pastori,
fasciati quinci e quindi d'alta grotta.*
(Dante, Purgatorio, Canto XXVII, 1472)

Such at that hour were we, all three of us,
I like the goat, and like the herdsman they,
Begirt on this side and on that by rocks.
(Dante, Purgatorio, Chant XXVII,
transl. Henry Wadsworth Longfellow, 1867)

1. INTRODUCTION

deer is an acronym for Dartmouth Environment Elementary Renderer; though there are already emulators of the Dartmouth OS (on which the original BASIC language environment ran), more complete and capable of running BASIC and ALGOL (see [1]), I wanted to build my own, and I wanted to do it in two languages:

- deer, the "operating system", in bash, due to its extreme ability to deal with strings (while lacking floating point numbers);
- dib, the BASIC interpreter, dia, the ALGOL interpreter and dif, the FORTRAN interpreter (to come), in C, for its speed and its math functions.

This was particularly interesting because I had to dive into a more complete bash and C theory before coding, and I was very pleased by that.

I spent one year (or so) in building deer, partly because of my need to study, and partly because I have a family that occupies my head and time, and leaves me night bits for coding. Finally the OS was ready in 2008 but it was an unfinished work. The BASIC side was completed the following year. The ALGOL side was completed much later, after finishing the Alpha version of taxi, my Algol interpreter based on the DEC system-10/20 ALGOL and on the ISO 1538-1984 (E) standard (completed in 2021) and adapted to the Dartmouth philosophy in 2026. The FORTRAN side was started as anew in 2026, after finishing the three precedent projects (deer, dib and dia).

It was not easy programming the first program in C (BASIC), because it is an algebraic language, and because my experience up till then was limited to stack languages, which are very easy to build. You can read the rest about this story in the file `dib_manual.pdf`.

2. THE DARTMOUTH OPERATING SYSTEM EMULATOR

deer is not a complete replica of the original Sixties OS. It's rather a subsystem, but as such it can create, edit, run, delete BASIC/ALGOL programs, and has some other features, along with minor differences compared to the original OS.

I discarded all the connection commands (network, kermit, and so on) and implemented only a sub-part of the documentation files (which couldn't all be applied to my work). There are no games, no web browsers, no mail readers. Nonetheless, you will be able to build and execute your own BASIC/ALGOL (and soon FORTRAN) programs. And some of the commands are full of humour, so i believe you could even laugh, which is not bad, nowadays.

To tell the truth, I'm conscious that this emulator goes beyond the real capabilities of the original OS; in all, deer is probably a subset of an Eighties OS version, rather than a Sixties one. In any case, I set it to make it work, to be *useful* rather than *faithful*; all the commands contained in [2] have been implemented, along with a bunch of others to ease the programmer's task.

2.1. Emulator options

The emulator must be run from console, using the following syntax:

```
$ deer <options>
```

Options are:

-h --help	Give this help and exit
-t n	Set auto shutoff to n seconds (n=0 disable shut-off)
-v --version	Show version and exit

A version of this brief list appears when `deer` is invoked as `deer -h/deer --help`.

Here are all options detailed:

`-t n`

If you do not type anything within 180 seconds during a `deer` session, the program terminates. You can set a different automatic shutdown value (in seconds) or disable the feature by specifying zero. It is a nod to the era when hardware was costly - not because it was costly per se, but because everything relied on mainframes with limited resources for all users. In the spirit of 1966!

2.2. installing deer

☞ **A foreword:** this is the only paragraph which uses the word *directory*, because it is referring to the Linux/UNIX file system. Throughout this text, the word *catalog* is used, as was commonly used in Dartmouth in the Sixties.

After unpacking the software, from within the unpacked directory, you have to open the file `makefile` and alter the `INSTALLDIR` variable with `your/bin/dir`, where `/your/bin/dir` is the directory of your `$HOME` where binaries can be run, and which is in `PATH`. If you don't change anything, the directory `$HOME/bin` will be used. After the customization, type:

```
$make install
```

and, if you want to install the man page:

```
$sudo make maninstall
```

if you have no root privileges.

The installation creates a directory called `.deer/` in the `$HOME` directory, along with its subdirectory `.explain/`, and copies the `EXPLAIN` screens; every time a new user number is used, a directory in `.deer` will be created with the first 6 characters of the user number turned to upper (e.g., if you login with `myra8697`, the directory `.deer/MYRA86` will be created); all subsequent logins with `MYRA86` will use the same directory, and different users will have different directories, so you may have many projects, each under a particular login name. Please note that the original DTSS accepted a login username made of six numbers or one letter and five any characters; `deer` is less restrictive, in this regard, and will accept any series of characters, cutting them to six and turning them to capitals.

2.3. Customizing the session

Before running `deer`, you can change some default variable values contained in the first lines of the script. They are:

deered

It contains the `deer` editor name, enclosed in double quotes. Default editor is `vim`, but you can use the one you prefer (`emacs`, `vile`, `nano` etc.), even a graphical one, if you want to.

INTERVAL

It contains the number of seconds for the autologoff procedure. Any value greater than 0 sets the option for that number of seconds (default is 180, which is about three minutes). If you put here 0, the timing feature is disabled.

DELAYBASE

It contains the delay factor for an empty cycle to be performed to slow down the listing with `LIST` and `DIFFERENCE`. On my former eMac G4 1.42 GHz, a value of 1,000 corresponded to about 2 lines per second (a speed comparable to the one of the old dear 8086). With my current computer, equipped with 4 Intel Core i3-5005U CPU (2.00GHz), and therefore about 8.5 times faster, I put here 8,500. You have the value to enter here at your disposal... perhaps 100,000? Does your super-fast number-cruncher computer deserve any less?

algol

(Important) It contains the name of the executable of an ALGOL compiler or interpreter of your choice. Of course, by default, `deer` comes with `dia` and you are not left alone in the choice...

basic

(Important) It contains the name of the executable of a BASIC compiler or interpreter of your choice. Of course, by default, `deer` comes with `dib`. I know that BASIC users are very fond on the language they use, so `dib` may perhaps appear even too simple; put here your wonderful BASIC executable name, with an advice... It must be able to interpret line numbers and capitalized commands... Free text solutions are too advanced for the DTSS...

sdebugopt

The default option is `-d` that in `dib` and `dia` activates the debug options. If your compiler has another option (for instance `--debug` or `-D`), put that option in this string.

UPPER

The capitalizing flag (1 by default); set it to zero if you want permanently have no capitalization at all. In this guise, using option `-l` will *enable* rather than disable the capitalization of the text.

Important: don't change anything else, unless you know what you're doing.

2.4. The session

When you run `deer`, the screen will be cleared, and a few data will be displayed on the upper left corner:

```
DARTMOUTH COLLEGE TIME SHARING, T1
29 JUN. 2026, AT 13:46, 32 USERS.
```

(You will see different time values and different logged users.) T1 means Tonibin 1, and is the machine simulated by `deer` (the original was D1, but I changed it).

At this point only three commands are available:

- HELLO to start a sequence to establish a connection to the system, obligatory to use the DTSS resources;
- HELP to print an informative screen for the use of EXPLAIN.
- BYE (also GOODBYE) to log off (there's an automatic log off if you don't type anything for 3 minutes, as explained in the options).

If at any point the system should hang, send an interrupt of type HUP INT PIPE QUIT TERM EXIT to the `deer` process. The interrupt is trapped, and before quitting, the current file will be saved to disc under the name `.dead.file` in the user catalog, both for programs or build files. You can recover this file (with the shell, from outside `deer`, in the directory `$HOME/.deer/XXXXX` where XXXXX is the user number) and see if what was saved has some features that deserve to be retained. Remember that the next interrupt will overwrite the dead file.

2.5. System commands

Notes before:

- 1) The definitions used in this manual are the following:
 - a PROGRAM file is a BASIC or ALGOL listing (with each line starting with a number); this kind of file may be executed.
 - a BUILD file is a textual file in free format; this kind of file cannot be executed; it's useful for writing documentation files.
 - CURRENT is the current file (PROGRAM or BUILD) in memory.
 - ARG is the (sometimes optional) argument of a command. It must be separated from its command by at least a white space
- 2) Any command (except HELP, HELLO, LIST - - and LISTNH) is recognized by means of the first three letters (so CAT and CATALYTIC will all act as CATALOG). But beware: you have to type HELLO, HELP and LIST - - in full to make them work.
- 3) The string WHAT? is printed if you type a command which is not recognized.
- 4) The same condensed information (or so) of the present paragraph can be obtained with HELP, or EXPLAIN HELP.
- 5) It's possible adding free text in PROGRAM mode, preceded by a line number. The information is stored and can be read at any time, but RUN will cause an error which depends on the SYSTEM in use. If instead the file is in

BUILD mode, every text is added to the BUILD file as free text, even text preceded by a line number.

- 6) There are some shortcuts understood by deer:
- the term *MYCAT is the name of the home catalog of the user with user number <usern> (located in OME/.deer/<usern>);
 - the term HOME is a shortcut for *MYCAT;
 - the term "." identifies the CURRENT catalog (and so it is quite useless);
 - the term .. identifies the upper father catalog.

Here's the commands explanations (text enclosed into {} is meant to be typed by the user).

ACCOUNT

Give some accounting information.

Note: original data were CRU, SECS, I/O, CORE, written on one line. I cannot fully replicate them in bash on a UNIX machine, and I changed the output to reflect more practical needs for the modern deer user, in two lines, the first with headers, the second with values.

The output looks like this:

CRU	SECS	I/O	CORE	CPU	LOGTIME	RUNS
54.18	40143.9	40	42467328	4.57	01:23:12	24

Data are:

- CRU¹ (Computer Resources Usage): it uses CORE, SECS and I/O to calculate a reasonable value for determining how much a mainframe session could cost.
- SECS² is the total in seconds of the usage of the CPU, a datum retrieved from /proc/stat, from the start of deer. It is not equal to the log time, which is much greater, because it's known that the CPU is waiting most of the time...
- I/O³: it expresses, through vmstat, the number of swap-in/swap-out instructions of the CPU.

¹ It was an abstract, cumulative metric introduced by the Dartmouth system administrators (particularly starting in the 1970s with the system's evolution on GE-635/Honeywell machines) to quantify overall system usage with a single number. The primary purpose of CRUs was billing: since DTSS was also sold to external schools and commercial enterprises via DTSS Inc., a summary unit of measurement was needed to determine a user's "load" - as charging solely for CPU seconds would have been unfair to users who saturated disk space or I/O channels. The CRU was a linear mathematical function (a weighted formula) that aggregated all resources consumed by a program or user session. The typical formula looked like this:

$$CRU = (A \times \text{CPU Seconds}) + (B \times \text{CORE Words} \times \text{Time}) + (C \times \text{I/O Operations}) + (D \times \text{Disk Storage})$$

where A, B, C, and D were multiplicative coefficients established by the computing center (the Kiewit Computation Center). Consequently: A program that used very little CPU but occupied a vast amount of RAM (CORE) incurred a high CRU cost. A very short program that performed massive I/O operations also incurred a high number of CRUs.

² In DTSS, SECS represented the actual CPU computation time (excluding wait times or user input) dedicated to executing programs. Since DTSS was a time-sharing system, the CPU would rapidly switch between users for fractions of a second. If a user wrote an infinite loop in BASIC and ran it, the terminal could remain occupied for a full minute of clock time while consuming only two actual seconds of processing time on the GE-235 CPU.

The most similar value is the CPU usage in seconds of my PC, a value that in UNIX and Linux systems is contained in /proc/stat, since start. The simulation is obtained considering that deer is a mainframe started in the precise moment you switched on your computer, and that using deer you simply gain access to your account in the mainframe.

³ On the Dartmouth Time Sharing System, the concept and visualization of I/O data differed structurally from those of a modern command like vmstat. While vmstat today calculates blocks per second read from or written to hard drives, on the DTSS, I/O data (often displayed to the user as usage statistics upon execution completion or in system logs) corresponded primarily to two fundamental elements of the era's architecture:

1. Terminal I/O (characters transmitted via Teletype). The DTSS was a pioneering system designed to allow dozens of users to connect simultaneously via Teletype (TTY) terminals. What it corresponded to: The primary I/O load monitored by the management computer (the DATANET-30) was the stream of characters (bytes) sent and received over telephone or telegraph lines. The modern parallel: This would be equivalent to monitoring network packets or characters exchanged during an SSH session.
2. Hard disk swapping operations (System I/O). The DTSS architecture relied on two computers: the DATANET-30 (which managed the terminals) and the GE-235 (which executed the programs). Both shared a large hard disk (with a capacity of approximately 16 million words). What it corresponded to: Because the GE-235's central memory (core memory) was tiny and could hold only one user program at a time for execution, the system performed continuous "swapping." It literally swapped a user's program out to the disk to load another into memory and grant

- **CORE**⁴ is the total, in **words**, of the free space in memory, a datum retrieved from the UNIX program **free**.
- **CPU**: it's the current occupation of the CPU; the complement to 100 expresses what is called *idle time*. This datum was not in the original DTSS and was added by me.
- **LOGTIME**: it expresses the total session time length since **deer**'s start. This datum was not in the original DTSS and was added by me.
- **RUNS**: it expresses the total of program executions (performed by **RUNs** and assuming this has some meaning for moderns systems). This datum was not in the original DTSS and was added by me.

APPEND

Add file {name} to the end of **CURRENT** file.

Usage:

```
{APPEND}  
APPENDING FILE NAME - - {name}
```

or

```
{APPEND name}
```

Notes:

- If no file name is given, the message **WHAT FILE?** appears and nothing is appended.
- If you **APPEND** a **BUILD** file to another, their content won't overlap and they will appear divided by an empty line, that can be removed by editing.
- If you **APPEND** a **PROGRAM** to another, the appended file will overwrite all lines whose numbers appear in both files (it's a merging operation, so you have to take care that the two files don't overlap).
- If you **APPEND** a **PROGRAM** to a **BUILD** file, it will appear with its line numbers as a listing queued to the **BUILD** file, which nonetheless remains a **BUILD** file, not executable. This may be helpful if you are building an informative document and want to add some code lines for explanation.
- You cannot **APPEND** a **BUILD** file to a **PROGRAM**. The message **NOTHING APPENDED.** appears when you try to do so.

BACKGROUND

The original **BACKGROUND** command initiated the running of a program or other activities under the control of the Background Monitor. Monitor activities were thus carried out without the necessity of the user terminal connected.

The Background Language⁵ was described in the "Background User's Manual" (technical Memorandum TM008).

In this emulated environment, this command is not implementable in the original sense. But you can use it: prepare to laugh!

BILL

Give billing information for the specified month. Prepare to laugh!

it a few milliseconds of CPU time. The system I/O metric measured this massive transfer of memory blocks to and from the disk. Modern parallel: It corresponds very closely to the **si** (swap-in) and **so** (swap-out) columns in **vmstat**, or to the heavy read/write activity caused by saturated RAM.

How was it quantified? If a user ran the **RUN** command on the DTSS, the system would return a line detailing resource usage upon completion. The I/O figure was not expressed in KB/s, but often in terms of the number of records or lines read from or written to the user's disk space (files saved via the **SAVE** command) and I/O channel time—specifically, the tenths of a second during which secondary processors (channel controllers) were occupied moving data on the user's behalf, thereby freeing up the main CPU.

You can understand that this tiny emulator cannot fully emulate this I/O apparatus. So I use **vmstat** and return the sum of **si** and **so** (swap-in and swap-out processes), and this is the value of I/O.

⁴ In the DTSS, **CORE** did not refer to "CPU cores" (a concept that did not exist in the 1960s) but rather to core memory (ferrite-core memory) - the ancestor of modern RAM. The Dartmouth system's memory was measured not in megabytes or gigabytes, but in words. The original GE-225/235 system had a total of only 16,000 words of memory. The "CORE" entry indicated to the user how many words - or what portion of that precious ferrite-core memory - their BASIC program was occupying to store code and variables.

⁵ I hadn't even imagined that such a language could exist. Dartmouth was a very cutting-edge University!

BUILD

Allow entering free text on **CURRENT** file. To terminate input press **RETURN** three consecutive times. After finished typing, giving a **SAVE** or a **REPLACE** command, will save the content to file as free text (not executable). Useful for writing documentation and various texts.

☞ **Warning!** If you type **BUILD** with an existing **PROGRAM** as **CURRENT**, its state will turn to **BUILD**, its content will be scratched and a subsequent **REPLACE** will overwrite the **PROGRAM** file. So, before calling **BUILD**, be sure to open or create a new **BUILD** file, or that the **PROGRAM** you're going to destroy is not necessary any more.

If you type **BUILD** after loading a **BUILD** file, the typed content will be added to the previous, with the intermission of a blank line. This lets you add text in successive times.

The correction of a **BUILD** file must be accomplished through editing (with **EDIT** or **QED**), because the **BUILD** technique lets you add text to a file, not review its content.

BRIEF

Typing **BRIEF** will cause subsequent timing, system message and headers to be suppressed; also, an asterisk (*) is printed instead of **READY** to notify you that you can type another command. See also the **NBRIEF** command.

BYE/GOODBYE

Terminate current session. If the **CURRENT** file has not been saved on disk yet, the system will invite you to do so by typing **SAVE** or **REPLACE**; this advice will appear only once; if you save the file or not, when you retype **BYE** you'll be allowed to exit anyhow, possibly losing the changes made (which may be exactly what you want).

CALCULATE

Parse an algebraic formula (in the same language according to **SYSTEM**), building a temporary file to pass to the compiler, suitable for **PRINT** in a **BASIC** program or **PRINT ()** in an **ALGOL** program.

Usage:

```
{CALCULATE <EXPR>}
```

Examples in **BASIC**:

```
CALCULATE 2 * LOG(2)
CALCULATE "VALUE = ";SIN(2.3)
CALCULATE "HELLO, WORLD!"
```

The same examples in **ALGOL**:

```
CALCULATE 2 * LN(2)
CALCULATE "VALUE = ", "", SIN(2.3)
CALCULATE "HELLO, WORLD!"
```

As you note, each print is in the shape understood by the relative interpreter, so that the three examples, that should produce the very same output, are equivalent.

☞ **Note:** This command is fairly inefficient, since it creates a mini **BASIC** or **ALGOL** program, calls the relative interpreter over it and then deletes the mini program. I implemented it this way to let a complete expression to be evaluated as if contained into a program. But the overhead in creating the file and executing it is responsible for the little delay in getting the answer (not counting the disk usage).

CATALOG

Give information about user's saved files in the **CURRENT** catalog or about another user in its proper catalog.

Usage:

{CAT}

for which the output is like this:

```
CAT
USER NO. 242424      PROBLEM NAME: WERTH    08 JUL. 2026    TIME: 08:52

CAPI                2 LINES
MATABA              22 LINES
PROVA               <BUILD>
SUBCAT/             <DIR>
TEST9               <EMPTY>
TESTBU              3 LINES
TEXT05              3 LINES
WERTH               <BUILD>

READY.
```

The BUILD files are labeled as such; the EMPTY label is for empty files (BUILD or PROGRAM). The dimension is reported in lines only for PROGRAM files.

The command CATALOG has some options:

- CATALOG CURRENT Get the file list (only names), turn the current file into a BUILD file (watch out! Everything in memory will be lost in) and save the list to the current file. Typing LIST will list the files, and with EDIT or QED the files names may be edited.
- CATALOG EXPLAIN Print the list of CATALOG options. The same information can be retrieved by EXPLAIN CATALOG from the general information center.
- CATALOG FILES List files names only in compact way.
- CATALOG LENGTH If you want to know the total occupation of files in one catalog, step into that catalog (using ENTER) and then type CATALOG LENGTH:

```
CAT LEN
HD 242424      20:04      28 JUL. 2026      1980 BYTES USED

CAPI          25
FORTRA        322
MATABA        417
NAMETY        67
PROVA         82
QAZ           54
QUERTY        50
QWER          57
SUBCAT/       541
TEST9         0
TESTBA       164
TESTBU        68
TEXT05        65
WERTH         68

READY.
```

As you can see, at the far right, the header reports the total of bytes used (included subdirectories), and the following file list, which is printed in alphabetical order, reports the dimension in bytes for each file. Sub-catalogs (identified by the / suffix) report the total dimension that they contain, including subdirectories. This is a different evaluation with respect to the standard CATALOG view, maybe nearer to the modern view.

- CATALOG RSIZE List files by dimension, larger last.

- **CATALOG SIZE** List files by dimension, larger first.

CREATE

Create a new catalog under **CURRENT**, with name {catname}. Usage:

```
{CREATE}  
CATALOG NAME--{catname}
```

or

```
{CREATE catname}
```

Directories are printed in **CATALOG** with a postfix / character.

DEBUG

Activate the debug on programs execution, enabling the -d option available for the program (in case your BASIC or ALGOL program has a different option for enabling the debugger, change the variable debugopt).

To enable debugging, type

```
DEBUG
```

To suppress debugging, type

```
DEBUG OFF
```

DIFFERENCE

Enhances the differences between the **CURRENT** and the argument file.

Usage:

```
{DIFFERENCE}  
DIFFERENCE FILE NAME--{name}
```

or

```
{DIFFERENCE name}
```

The output is like this:

```
DIFF  
DIFFERENCE FILE NAME--OTHER  
  
USER NO. 242424      PROBLEM NAME: MATABA    09 JUL. 2026      TIME: 06:36  
  
CURR: 5 PRINT "EXAMPLE OF MAT STATEMENTS USAGE"  
FILE: 40 MAT READ B,A  
CURR: 40 MAT READ A,B  
FILE: 90 MAT C=A*B  
CURR: 90 MAT C=B*A  
FILE: 167 PRINT "INVERSE MATRIX"  
FILE: 175 PRINT "MATRIX BY ITS INVERSE"
```

The output marks the **CURRENT** file (the one in memory, MATABA in the example) with **CURR** and the second file (the one on disk, OTHER in the example) with **FILE**. If both line numbers are present for **CURR** and **FILE**, differences are in the text. Otherwise, if the line is only for either, the other has no lines with that line number. The file in memory retains its path, so that you can compare against it a file which is in another catalog you reached with **ENTER**.

Note that this command, like **LIST**, is affected by the delay, that you can alter with **SPEED**.

EDIT

Start editing a file with the builtin line editor. This command acts only on **BUILD** files.

Usage

{EDIT}

to act on current **BUILD** file, or

{EDIT NAME}

to act on **BUILD** file **NAME**; if **CURRENT** is the edited file, it will inherit all the changes; if **CURRENT** is a **PROGRAM** it will remain untouched. When the editor starts, you will see the prompt, which is different from the **DTSS** prompt:

>

If a command requires one or two numeric values, they must be specified first and separated from the command by at least one space.

The editor commands are:

Command:Scope	
a	add one line after last; the prompt is :
b	add one line before first; the prompt is :
<n> d	delete line <n> (e.g. 3 d); all lines starting from <n>+1 are shifted downwards.
<n> e	edit line <n> (e.g. 4 e); the editing is made easier by printing the line, where the user can move the cursor inside, and change what must be changed. When the user presses Enter, the new line is saved in memory (not on disc); if instead the user presses CTRL-U followed by Enter, the line is not saved in memory and the original line is maintained.
h	print a short help screen.
<n> i	insert one empty line at position <n> (e.g. 3 i); all lines starting from <n>+1 are shifted upwards and so a new empty line is left at position <n>, suitable for a command like <n> e to edit that line.
l	list the entire file (the one in memory not the one on disc).
<n1> <n2> l	list file (the one in memory not the one on disc) from line <n1> to line <n2>.
n	list the entire file (the one in memory not the one on disc) with line numbers.
<n> p	print line <n>.
q	quit without saving; Watch out! If, before typing q, you don't save to disc, all changes are lost.
r	reset memory. This means the whole cancellation of the memory, but not of what is stored on disc.
s	save memory to disc; if CURRENT is a BUILD file, the memory is saved also on the temporary file in use by the BUILD file.

ENTER

Change **CURRENT** catalog to catalog {catname}.

Usage:

```
{ENTER}  
CATALOG NAME--{catname}
```

or

```
{ENTER catname}
```

The command ENTER interprets the figures . . , HOME and *MYCAT.

☞ **Warning:** This command can jump from one catalog one step down or one step up, so to jump, for instance, two steps down, the command must be given twice. An exception is ENTER HOME which jumps instantaneously to the user home catalog.

EOF

The command EOF is not a command, it is rather a particular token that is intercepted in the routine that waits for a DTSS command and terminates immediately the execution if EOF is typed. As such, it may not be available if the system is stuck in an endless loop (in this case, CTRL -C is perhaps the solution). In all other cases, typing EOF is the equivalent of a soft CTRL -C, but remember that this operation does not permanently save the CURRENT file in memory.

EXECUTE

Execute the program specified as argument in the CURRENT catalog. Only programs can be executed; if you try to execute a BUILD file, the error message "ILLEGAL LINE NUMBER" will be printed.

EXIT

A synonym of BYE. This was not in the original DTSS.

EXPLAIN

Using the command, you could obtain on the original DTSS information on various aspects of the Dartmouth Time-Sharing System, such as the history of the system, the building, numbers to dial to access the system, and sources of information at Kiewit. For all subjects devoted to software not available and society topics, humorous text was written to delight the user. So prepare to laugh!

This emulation has retained all the information not directly related to Kiewit. Typing EXPLAIN without arguments, will cause printing an introductory help screen. Typing EXPLAIN TOPICS will yield a list of topics and their corresponding subject matter.

The Algol errors (which are not textual) are explained; see EXPLAIN ALGERR or type directly the specific error with EXPLAIN E07 (for instance).

Each reference should be consistent (if you are told to use EXPLAIN MISTAKES, typing EXPLAIN MISTAKES will show you the relative help screen) but if it's not so, please write me.

HELLO

Start a sequence of questions for logging in. Here's an example of output:

```
{HELLO}  
52.38 CRU  
  
USER NUMBER--{MYRA86}  
SYSTEM--{BASIC}  
NEW OR OLD--{OLD}  
OLD PROBLEM NAME--{LINEAR}  
READY.
```

Notes:

- The **USER NUMBER** - - question requires an answer; in pressing Enter and returning an empty value, the question is asked again until an answer is given.
- The **SYSTEM** - - question requires an answer; in pressing Enter and returning an empty value, the system **BASIC** is assumed.
- If you press Enter on the question **NEW OR OLD** - -, this will be interpreted as **OLD**.
- If you press Enter on the question **NEW PROBLEM NAME** - -, the system will respond with the string:

NOT A REGULAR FILE. SETTING DEFAULT NAME.

The default name is **FILE04**.

- If you press Enter on the question **OLD PROBLEM NAME** - -, the system will respond with the string:

NOT A REGULAR FILE. SETTING DEFAULT NAME.

The default name is **FILE06**.

The command **HELLO** may be typed at any point when we want to change the user name.

HELP

Print a help screen which introduces the user to the help system (through **EXPLAIN**).

HOME

Shortcut for **ENTER *MYCAT**; re-enters main user catalog. It must be pointed out that **ENTER HOME** is the same, so that the three commands perform the same task

IGNORE

Retrieve saved file, losing all changes made.

JOB

This is a synonym for **STATUS**.

LENGTH

Give length of **CURRENT** file.

LIST

List **CURRENT** file (**BUILD** or **PROGRAM**) in its entirety. If a numeric argument is specified, in case of a **PROGRAM** file, only the specified line is printed, without header and **READY**. (this last is a **deer** addition not available in the original DTSS). E.g.

LIST 70

70 PRINT "MATRIX B"

LISTNH

List **CURRENT** file (**BUILD** or **PROGRAM**) in its entirety without the header and without the final **READY**. Useful to produce a fast copy & paste listing to be put in documentation or printed.

LIST - -XXXXX

List **CURRENT** file starting from line **XXXXX** up to the end. Mind the two dashes following **LIST**.

Notes:

- If **XXXXX** is not specified, a normal **LIST** is performed.
- If **LIST - -** is applied on a **BUILD** file, **LIST** is performed anyway, regardless of the argument **XXXXX**.

NBRIEF

Typing **NBRIEF** terminates the brief mode operation and returns your terminal to the normal mode. This will cause the restoring of the system messages and headers, and **READY**. is printed, as usual, instead of the brief mode

asterisk. See also the **BRIEF** command.

NEW

Create a new empty file and set it **CURRENT** with name `{name}` and state **PROGRAM** (you can safely type **BUILD** after **NEW**, because the file is empty).

Usage:

```
{NEW}  
NEW PROBLEM NAME--{name}
```

or

```
{NEW name}
```

If `{name}` is empty, the name **FILE04** is given.

OLD

Retrieve an existing file with name `{name}` and set it **CURRENT**; the state will be **PROGRAM** if the first character of the file is a number (supposing it is a line number), otherwise it will be a **BUILD** file.

Usage:

```
{OLD}  
OLD PROBLEM NAME--{name}
```

or

```
{OLD name}
```

If `{name}` is empty, the name **FILE06** is given.

OPTION

Pass its argument to the compiler. The argument constitutes the entire set of options to be passed to the compiler; therefore, they must be options that can be precisely interpreted. **deer** performs no checks on these options (regarding their existence, correctness, completeness, or otherwise), so it is the user's responsibility to know which options to use.

Usage

```
{OPTION}  
OPTIONS--{XXXX}
```

There is only one form of the command; the `{OPTION XXXX}` version would theoretically be possible, but since the text is always converted to uppercase in this case, the same would happen to the options, effectively rendering them unusable - given that almost all compilation options for an interpreter or compiler are lowercase.

PERFORM

Takes commands from a textual file and executes them in turn, as if typed from the keyboard.

Usage

```
PERFORM  
PERFORM FILE NAME--{file}
```

or

```
PERFORM {file}
```


The commands executed by **PERFORM** do not print **READY** after each execution but only at the end of **PERFORM**.

PUNCH

Send **CURRENT** file to printer; Usage:

{**PUNCH**}

Send file {**name**} (which may be different from **CURRENT**) to printer: usage:

{**PUNCH name**}

Notes:

- If the file has zero bytes (an empty file), the act of sending it to the printer will end in nothing. No error is printed, no file is sent to the printer.
- In order to make this command work, your Linux/UNIX box should be connected to a printer suitable for **lpr**.

QED

Start editing **CURRENT** saved file with the user editor (specified in the variable **deered**). When exiting from the quick editor, the file must be saved, in order to see the changes on **deer**.

This command acts both on **PROGRAMS** and on **BUILD** files.

RENAME

Change name of **CURRENT** file to {**name**}, retaining the state (**PROGRAM** or **BUILD**).

Usage:

{**RENAME**}
RENAME PROBLEM NAME - - {name}

or

{**RENAME name**}

REPLACE

Save **CURRENT** file to disc, replacing previous contents.

RUN

Execute **CURRENT PROGRAM** file; only language programs can be run, and **BUILD** files cannot be run. If you try, the message "**NO PROGRAM.**" is printed (this command won't be counted as a successful command in the count of **ACCOUNT**).

It must be clear that the file that is run is the one in memory⁶. This function allows the user to temporarily modify certain lines of code and verify the effect without interfering with the program saved on disk. If the user wishes to keep the changes, **REPLACE** may be used; if, on the other hand, the user wants to discard them, **IGNORE** may be used.

SAVE

Store **CURRENT** file to disk if file does not exist (if file exists, a warning will appear; in this case use **REPLACE**).

SCRATCH

This command erases the **CURRENT** version of your file, retaining the state (**PROGRAM** or **BUILD**). Saved files are not affected by this command; the current file name is not changed.

⁶ In previous versions of **deer**, a **SAVE/REPLACE** was executed before **RUN**; this was done in the erroneous belief that this was done also on the 1964 manual, where it's written "*The same was applied in the 1964 OS (see the October 1964 BASIC manual for examples in which a typed program is directly followed by a RUN)*". But of course this was not entirely true, because you can **RUN** a program that is not saved, because it is in memory. So, in this version, I want to divide the running phase (performed saving the program to a temporary file) from the saving phase (which the user may not want),

SORT

Sort contents of the **CURRENT BUILD** file; the original DTSS could also sort a **PROGRAM**, but this is not necessary in **deer**, because the file is kept always sorted automatically in memory and during saving, that's why for a **PROGRAM** sorting is avoided.

SPEED

Change the speed of the delay for **LIST** and **DIFFERENCE**.

Usage:

SPEED {num}

The delay was introduced to make **deer** more conforming to the speed that was in the Mid Sixties. But of course, the user may want use the speed available on his computer; the value {num} may be one of the following:

- if {num} = 0, the smaller value 1 will be set. In this way, the speed is the highest.
- if {num} = 1..10, the value will be set according to the following formula:

$$\text{newspeed} = \frac{\text{currspeed}}{11 - \text{num}}$$

to obtain a fine granularization of the value in ten steps, from the faster 1 to the slower 10.

The speed set with **SPEED** will remain until a new speed definition will occur.

STATUS

When an user submitted his job to the Background Monitor, with **BACKGROUND**, he could later check the status of the request with **STATUS**. The term **JOB**, on the other hand, was more common in IBM systems (under the control of JCL, or Job Control Language) or in batch systems, where it specifically identified the "job" object submitted to the computer. So in this environment, both terms may be used.

This command is coordinated with the last **BACKGROUND** command, so prepare to laugh (launch **BACKGROUND** and then **STATUS**).

SYSTEM

Change system language to {name} (actually, only **BASIC**, **ALGOL** and **FORTRAN** are accepted).

Usage:

{SYSTEM}
SYSTEM NAME - - {name}

or

{SYSTEM name}

After that, the **NEW OR OLD** - - question is asked, for selecting an appropriate file for the chosen system.

TERMINAL

Print the state of your terminal, emulated or true TTY. Internally, **deer** uses the UNIX program **tty** to retrieve the hardware identification.

UNSAVE

Delete **CURRENT** file from disk. The memory content is preserved.

Usage:

{UNSAVE}

Delete a file or a catalog from disk. Usage:

{UNSAVE name}

- 1) **CAUTION!** Don't use this command carelessly! Remember that, if you type UNSAVE, deer ****WON'T ASK**** you for a file name; rather, **IT WILL DELETE CURRENT FILE!** If you want to delete a different file, ****YOU HAVE**** to type its name after UNSAVE!
- 2) If used with catalogs (directories), UNSAVE will delete the catalog ****AND ALL ITS FILES****! Nothing is asked, nothing is written.

In all, use this command with **GREAT CARE!**

UPDATE

If you created a catalog, and you want to move one or more files to it, you can use the UPDATE command, to be applied on one file only.

Usage:

```
{UPDATE}  
PROBLEM NAME -- {name}  
CATALOG NAME -- {catname}
```

or

```
{UPDATE name}  
CATALOG NAME -- {catname}
```

or

```
{UPDATE name catname}
```

After the command is issued, the file {name} is moved to catalog {catname}.

USERS

Give number of currently logged users. The output value is a random number to give you the impression the system is not used by you only!

VIEW

Show CURRENT or argument file without editing it.

Usage:

```
{VIEW}
```

to view CURRENT file, and

```
{VIEW name}
```

to view a file in some catalog.

WHAT

Give information about the current session, in the form:

```
CURRENT FILE  MOTION  
FILE TYPE     PROGRAM  
FILE STATE    SAVED  
COMPUTER      T1 (deer system)  
USER NUMBER   242424  
SYSTEM        BASIC  
SEARCH PATH   :242424/SUBCAT/
```

The FILE TYPE may be PROGRAM or BUILD, the FILE STATE may be SAVED or UNSAVED. The SEARCH PATH includes the current path where files are searched.

Notes after:

- 1) After any command performance, successful or not, READY is printed so that you know you can type the next command.
- 2) Please, pay attention to the UNSAVE command: if you happen to type UNSAVE inadvertently, you can recover the file (because it's still in memory) by typing SAVE immediately after it. File will be rewritten.
- 3) The SAVE and REPLACE commands play a similar role: the first must be used the first time you write a file to disk; the second must be used when you want to replace what's stored on disk with what's on memory, updating file's contents⁷.
- 4) The command EXIT has been created to offer a helping hand to shell users who want to terminate the job. It behaves the same way as BYE.
- 5) The command ? behaves like CALCULATE, as a homage to the Commodore 64 BASIC users that will approach deer.

2.6. Tips on file manipulation

In the following, I assume that:

<home>
is the catalog where we are

<cat>
is the destination catalog

<cur>
is the name of current file

<name>
is a new name for current file or a name of an existing file other than current

Anywhere I write SAVE you must read SAVE or REPLACE, depending on the file existence.

How to move CURRENT file to another catalog

```
UNSAVE
ENTER <cat>
SAVE
```

This procedure is not safe, because if something happens to your system between UNSAVE and SAVE the file is lost, though it's very unlikely since it's question of a few seconds; in any case, if you want a safer (and longer) procedure:

```
ENTER <cat>
SAVE
ENTER . .
UNSAVE
```

The command UPDATE has been added to simplify the procedure.

How to rename an existing file

```
UNSAVE
RENAME <name>
SAVE
```

A safer procedure:

```
RENAME <name>
SAVE
OLD <cur>
```

⁷ The modern approach uses a single operation (SAVE), but the dual nature of the mid-1960s DTSS saving operations offers food for thought regarding which operations were considered important and the cost of memory - something we take for granted today: SAVING was one thing (it meant occupying memory), while REPLACING was another (memory consumption was negligible in most cases).

UNSAVE

How to move and rename a file to another catalog

```
UNSAVE
ENTER <cat>
RENAME <name>
SAVE
```

A safer procedure:

```
ENTER <cat>
RENAME <name>
SAVE
ENTER ..
OLD <cur>
UNSAVE
```

How to copy a file in the same catalog (renaming it)

```
RENAME <name>
SAVE
```

How to copy a file in another catalog with the same name

```
ENTER <cat>
SAVE
```

How to copy a file in another catalog with another name

```
ENTER <cat>
RENAME <name>
SAVE
```

How to delete a file

```
OLD <name>
UNSAVE
SCRATCH
```

or

```
UNSAVE <name>
```

3. CONCLUSIONS

deer is an emulation, written in bash: don't expect it to be fast, precise, robust. After all it's a bit more than a game. And remember that `dib` and `dia`, the BASIC and ALGOL interpreters, may be used alone, as UNIX programs, and in this case `deer` may be forgotten.

But, please, if you want to help me improve this program, write me tips and snippets of code to better it or make it compatible with other UNIX systems.

I hope you like deer.

It's GPL: Enjoy! ♥

4. REFERENCES

The following documents were very useful to understand basic principles and set up general behaviors:

- [1] The dtss simulator, with many information about the system, can be found at <http://www.dtss.org/> - in the site you can also find a link to an emulator written in java and running online (a registration is requested in order to use it - I didn't).
- [2] "Dartmouth BASIC manual" for version II, October 1964, available as a pdf file at https://www.bitsavers.org/pdf/dartmouth/dtss/196410_BASIC.pdf
- [3] "Programming with BASIC", by Byron S. Gottfried, edition 1, 1975 (Italian version, titled "Programmare in BASIC", Schaum 1982).
- [4] "Dartmouth Time-sharing System - a brief description", from October 1964, outlines the system in which the OS was implemented, and is available as a pdf file at https://www.bitsavers.org/pdf/dartmouth/dtss/196410_DTSS_descr.pdf

Table of Contents

1.	Introduction	5
2.	The Dartmouth Operating System emulator	5
2.1.	Emulator options	5
2.2.	installing deer	6
2.3.	Customizing the session	6
2.4.	The session	7
2.5.	System commands	7
2.6.	Tips on file manipulation	20
3.	Conclusions	22
4.	References	23